17. Illustrate the Deadlock Avoidance Concept by Simulating Banker's Algorithm Using C

Code - Part 1 (Lines 1–35)

```c
1  #include <stdio.h>
2
3  #define MAX_PROCESSES 5
4  #define MAX_RESOURCES 3
5
6  int is_safe();
7
8  int available[MAX_RESOURCES] = {3, 3, 2};
9
10 int maximum[MAX_PROCESSES][MAX_RESOURCES] = {
11     {7, 5, 3},
12     {3, 2, 2},
13     {9, 0, 2},
14     {2, 2, 2},
15     {4, 3, 3}
16 };
17
18 int allocation[MAX_PROCESSES][MAX_RESOURCES] = {
19     {0, 1, 0},
20     {2, 0, 0},
21     {3, 0, 2},
22     {2, 1, 1},
23     {0, 0, 2}
24 };
25
26 int request_resources(int process_num, int request[]) {
27
28     // Check if request can be granted
29     for (int i = 0; i < MAX_RESOURCES; i++) {
30         if (request[i] > available[i] ||
31             request[i] > (maximum[process_num][i] - allocation[process_num][i])) {
32             return 0;
33         }
34     }
35
```

Code - Part 2 (Lines 36–70)

```
36      // Temporary allocation
37      for (int i = 0; i < MAX_RESOURCES; i++) {
38          available[i] -= request[i];
39          allocation[process_num][i] += request[i];
40          maximum[process_num][i] -= request[i];
41      }
42
43      // Check safe state
44      if (is_safe()) {
45          return 1;
46      } else {
47          // Rollback
48          for (int i = 0; i < MAX_RESOURCES; i++) {
49              available[i] += request[i];
50              allocation[process_num][i] -= request[i];
51              maximum[process_num][i] += request[i];
52          }
53          return 0;
54      }
55  }
56
57  int is_safe() {
58      int work[MAX_RESOURCES];
59      int finish[MAX_PROCESSES] = {0};
60
61      // Copy available to work
62      for (int i = 0; i < MAX_RESOURCES; i++) {
63          work[i] = available[i];
64      }
65
66      int count = 0;
67
68      while (count < MAX_PROCESSES) {
69          int found = 0;
70
```

Code - Part 3 (Lines 71–105)

```
71          for (int i = 0; i < MAX_PROCESSES; i++) {
72
73              if (finish[i] == 0) {
74                  int j;
75
76                  for (j = 0; j < MAX_RESOURCES; j++) {
77                      if ((maximum[i][j] - allocation[i][j]) > work[j])
78                          break;
79                  }
80
81                  if (j == MAX_RESOURCES) {
82                      for (int k = 0; k < MAX_RESOURCES; k++) {
83                          work[k] += allocation[i][k];
84                      }
85
86                      finish[i] = 1;
87                      found = 1;
88                      count++;
89                  }
90              }
91          }
92
93          if (found == 0) {
94              return 0; // Unsafe
95          }
96      }
97
98      return 1; // Safe
99  }
100
101 int main() {
102     int process_num;
103     int request[MAX_RESOURCES];
104
105     printf("Enter process number (0 to 4): ");
```

Code - Part 4 (Lines 106–120)

```
106     scanf("%d", &process_num);
107
108     printf("Enter resource request (e.g., 1 0 2): ");
109     for (int i = 0; i < MAX_RESOURCES; i++) {
110         scanf("%d", &request[i]);
111     }
112
113     if (request_resources(process_num, request)) {
114         printf("Request granted.\n");
115     } else {
116         printf("Request denied. System is not in safe state.\n");
117     }
118
119     return 0;
120 }
```

Output

```
Enter process number (0 to 4): 1
Enter resource request (e.g., 1 0 2): 1 0 2
Request granted.


...Program finished with exit code 0
Press ENTER to exit console.
```